Libft : correction des questions et présentation du projet

Oct 21, 2026 · @mostafa

Règles générales

Une erreur sur l'une de ces règles peut valoir 0 avant même que l'évaluateur lise ton code.

- **Q1. Compilation.** Le compilateur est `cc`, avec les flags `-Wall -Wextra -Werror`.
- **Q2. Règles du Makefile.** `$(NAME)`, `all`, `clean`, `fclean` et `re`, plus `bonus` si tu soumets des bonus.
- **Q3. Pas de relink inutile.** Si rien n'a changé, `make` ne refait rien : les `.o` dépendent des `.c` et du header, et l'archive dépend des `.o`.
- **Q4. Norme.** Une erreur de norme, même dans un fichier bonus, donne 0.
- **Q5. Crashs.** Un segfault, un bus error ou un double free rend le projet non fonctionnel et donne 0, sauf si le crash vient d'un comportement indéfini.
- **Q6. Emplacement.** Tous les fichiers sont à la racine du dépôt, et `libft.a` est créé à la racine.

Contraintes techniques

Quatre interdictions et obligations qui structurent tout le code de la bibliothèque.

- **Q7. Variables globales.** Elles sont strictement interdites.
- **Q8. Fonctions auxiliaires.** On les déclare `static`, ce qui limite leur portée au fichier.
- **Q9. Création de la bibliothèque.** On utilise `ar`. `libtool` est interdit.
- **Q10. `restrict` et `-std=c99`.** `restrict` appartient à la norme C99 : le sujet interdit de l'inclure dans tes prototypes et de compiler avec `-std=c99`.

Partie 1 : fonctions libc

Les pièges se trouvent dans les cas limites : chevauchement mémoire, troncature, caractère `'\0'` et conversion en `unsigned char`.

- **Q11. Fonctions `is*`.** `isalpha`, `isdigit`, `isalnum`, `isascii` et `isprint` retournent 1 si le caractère appartient à la classe testée, 0 sinon (pas n'importe quelle valeur non nulle).
- **Q12. `memcpy` et `memmove`.** `memcpy` suppose que les zones ne se chevauchent pas, sinon le comportement est indéfini. `memmove` gère le chevauchement : si `dst` est avant `src` on copie du début vers la fin, sinon de la fin vers le début.
- **Q13. `calloc` avec 0.** Si `nmemb` ou `size` vaut 0, `calloc` retourne un pointeur unique qui peut être passé à `free()`. Vérifie aussi le dépassement de `nmemb * size`.
- **Q14. `strlcpy`, `strlcat`, `bzero`.** glibc ne les fournit pas par défaut. Pour comparer avec la version système, inclus `<bsd/string.h>` et compile avec `-lbsd`.
- **Q15. Valeur de retour de `strlcpy` et `strlcat`.** `strlcpy` retourne la longueur de `src`. `strlcat` retourne la longueur initiale de `dst` (bornée par `size`) plus celle de `src`. Si le résultat est supérieur ou égal à `size`, il y a eu troncature. Les deux copient au plus `size - 1` caractères et terminent par `'\0'`.
- **Q16. `strchr` et `'\0'`.** Chercher `'\0'` retourne un pointeur vers le `'\0'` final de la chaîne. Le caractère cherché est converti en `char`.
- **Q17. `strncmp` et `memcmp`.** `strncmp` s'arrête au `'\0'` et compare au plus `n` caractères. `memcmp` compare exactement `n` octets sans s'arrêter au `'\0'`. Dans les deux cas la comparaison se fait en `unsigned char` et le résultat est la différence des premiers octets différents.

Partie 2 : fonctions additionnelles

Chaque fonction alloue avec `malloc` : les cas limites et la libération en cas d'échec sont ce que l'évaluateur teste.

- **Q18. `ft_substr`.** Si `start` est supérieur ou égal à la longueur de `s`, on retourne une chaîne vide allouée (pas NULL). Si `len` dépasse ce qui reste, on tronque à ce qui reste : on alloue donc `min(len, reste) + 1`.
- **Q19. `ft_strtrim`.** On avance depuis le début tant que le caractère est dans `set`, puis on recule depuis la fin de la même façon. On copie ce qui reste au milieu, et si tout est dans `set`, on retourne une chaîne vide.
- **Q20. `ft_split`.** Pour une chaîne vide ou composée uniquement de séparateurs, on retourne un tableau alloué contenant seulement le NULL final. Les séparateurs consécutifs ne produisent pas de chaînes vides. Si une allocation échoue, on libère toutes les chaînes déjà allouées, puis le tableau, et on retourne NULL.

- **Q21. `ft_itoa`.** Cas à gérer : `INT_MIN` (on ne peut pas faire `-n` en `int`, donc on passe par un type plus large ou on traite chiffre par chiffre), `0` (qui donne `"0"`) et les négatifs (signe `-`). Calcule la taille avec le signe et le `'\0'`.
- **Q22. `ft_strmapi` et `ft_striteri`.** `ft_strmapi` crée et retourne une nouvelle chaîne, et `f` reçoit (index, caractère) et retourne un `char`. `ft_striteri` modifie la chaîne en place, ne retourne rien, et `f` reçoit (index, adresse du caractère).
- **Q23. Fonctions `ft_put*_fd`.** Seule la fonction externe `write` est autorisée.
- **Q24. `INT_MIN` dans `ft_putnbr_fd`.** Soit tu traites `INT_MIN` comme cas particulier (écriture directe de la chaîne), soit tu utilises une récursion sur `n / 10` et `n % 10` avec un type plus large, soit tu travailles avec des valeurs négatives pour ne jamais avoir à nier `INT_MIN`.

Partie 3 : listes chaînées

Tout repose sur deux idées : le double pointeur pour modifier la tête de liste, et le callback `del` pour libérer un contenu de type inconnu.

- **Q25. `t_list`.** `content` est un `void *` et `next` pointe vers le nœud suivant (NULL pour le dernier). `void *` permet de stocker n'importe quel type de donnée.
- **Q26. Double pointeur dans `ft_lstadd_front` et `ft_lstadd_back`.** La tête de liste peut changer : il faut modifier le pointeur de l'appelant, donc recevoir son adresse.
- **Q27. `ft_lstadd_back` sur liste vide.** Si `*lst` est NULL, la liste est vide et `new` devient la tête : `*lst = new`.
- **Q28. `ft_lstdelone` et `ft_lstclear`.** `ft_lstdelone` libère un seul nœud (contenu via `del`, puis le nœud) sans toucher au suivant. `ft_lstclear` libère le nœud donné et tous ses successeurs, puis met `*lst` à NULL, d'où le double pointeur. Il faut sauvegarder `next` avant de libérer chaque nœud.
- **Q29. Échec d'allocation dans `ft_lstmap`.** On libère la nouvelle liste déjà construite avec `ft_lstclear` et `del`, sans oublier le contenu que `f` vient de créer pour le nœud qui a échoué. On retourne ensuite NULL. La liste d'origine reste intacte.
- **Q30. Pourquoi un paramètre `del`.** La bibliothèque ne sait pas comment `content` a été alloué (chaîne, structure, entier, rien du tout). C'est à l'utilisateur de fournir la fonction de libération adaptée : c'est le principe du callback, qui rend la liste générique.

README et évaluation

Le README et la modification demandée en défense vérifient que tu comprends ce que tu as rendu.

- **Q31. Première ligne du README.** Elle est en italique : *This project has been created as part of the 42 curriculum by <login1>[, <login2>[...]].*
- **Q32. Sections minimales.** Description, Instructions et Resources, plus une description détaillée de la bibliothèque. L'anglais est recommandé.
- **Q33. Usage de l'IA.** La section Resources doit préciser pour quelles tâches et quelles parties du projet l'IA a été utilisée.
- **Q34. Modification du projet en défense.** C'est une petite modification demandée pendant l'évaluation (changer un comportement, réécrire quelques lignes), faisable en quelques minutes. Pour t'y préparer, sois capable de réécrire chaque fonction de tête, de l'expliquer, et connais bien tes propres tests.

Qu'est-ce que Libft ?

Libft est le premier projet du tronc commun de l'école 42 : tu y construis ta propre bibliothèque statique en C, `libft.a`, qui réunit trois familles de fonctions.

- Des réimplémentations de fonctions de la libc (`strlen`, `memcpy`, `atoi`, etc.).
- Des fonctions utilitaires (`ft_split`, `ft_itoa`, `ft_strjoin`, etc.).
- Une petite bibliothèque de listes chaînées.

Pourquoi 42 donne ce projet

Libft sert à construire des fondations solides en C, et à te donner une boîte à outils que tu réutilises dans la suite du cursus.

1. **Comprendre les fonctions standard.** En réécrivant `memmove` ou `strlcat`, tu découvres les cas limites, les chevauchements de mémoire, les troncatures et les comportements indéfinis, que tu ne vois pas en utilisant ces fonctions comme des boîtes noires.
2. **Maîtriser les bases du C.** Pointeurs, chaînes terminées par `'\0'`, allocation dynamique avec `malloc` et `free`, fuites mémoire, pointeurs de fonctions (`strmapi`, `lstmap`) et doubles pointeurs (listes).

3. **Apprendre la rigueur.** La norme de 42, les flags `-Wall -Wextra -Werror`, zéro crash et zéro fuite : ce sont des habitudes de développeur professionnel.
4. **Apprendre l'outillage.** Écrire un Makefile correct sans relink inutile, créer une archive avec `ar`, organiser un dépôt Git et tester son propre code.
5. **Te constituer une boîte à outils.** Dans `get_next_line`, `ft_printf`, `push_swap` ou `minishell`, les fonctions autorisées sont limitées et tu peux souvent utiliser ta propre libft. Plus elle est solide, plus tu gagnes de temps ensuite.
6. **Pratiquer l'évaluation par les pairs.** Tu dois expliquer ton code à un autre étudiant, ce qui vérifie que tu l'as réellement compris, d'où la modification du projet en défense.

Le sujet insiste sur le fait que le parcours compte plus que le résultat. Si tu copies du code sans le comprendre, tu seras bloqué en défense puis à l'examen, où il n'y a pas d'IA. Utilise donc ces questions comme auto-évaluation : réécris chaque fonction de mémoire, explique-la à voix haute, puis compare avec ton code.